

THE FUJINET BUILD CONTAINER

defoogi

DEMYSTIFIED

One container, every FujiNet toolchain — and what it would take to build them all yourself.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top. The text inside shows the output of a 'make' command in a container named 'defoogi'.

```
wario@defoogi:~/my-app$ defoogi make
cc65  →  apple2 · c64 · atari · nes
cmoc  →  tandu coco (6809)
zcc   →  msx · coleco adam (z80)
wcl   →  ibm pc / dos (x86)
atr/ac →  disk image, mounted by fujinet
✓ build complete — artifacts owned by you
wario@defoogi:~/my-app$
```

Contents

1 • What defoogi Is, and the Problem It Solves	3
The problem it solves	3
The headline feature: ownership preservation	3
2 • How defoogi Is Built (Architecture)	4
The three core stages and the components	4
The base image — <code>head.docker</code>	4
The clever part — build once, package as a <code>.deb</code>	4
Assembly — the <code>Makefile</code> streams everything into one Dockerfile	5
Reproducibility — <code>versions.env</code>	5
Multi-arch	5
3 • How defoogi Runs (The Runtime Model)	6
<code>start</code> → installed as <code>defoogi</code>	6
<code>cntnr-init</code> — the ownership-preservation magic	6
<code>defoogi-make</code> — directory-aware <code>make</code>	6
4 • The Complete Toolchain Inventory	8
Cross-compilers (high-level languages → retro CPUs)	8
Assemblers	9
Disk-image & file tools — <i>making the target media</i>	9
Embedded / FujiNet-hardware toolchains	9
Base system, libraries & build dependencies (the invisible glue)	10
Declared-but-not-built (housekeeping note)	10
5 • Installing Each Tool Yourself (The Hard Way)	11
cc65 (6502)	11
CMOC + lwtools (6809)	11
z88dk + SDCC (Z80)	11
Open Watcom v2 (x86 / DOS)	12
MADS + Mad Pascal (Atari 6502)	12
Atari disk tooling, atari-tools, cc1541	12
AppleCommander (Apple II disks)	12
cpmtools / mtools / nasm	12
PlatformIO + Pico SDK (embedded)	13
The DIY pain summary	13
6 • Why defoogi Is the More Complete Option	14
7 • Using defoogi Day-to-Day	15
Install	15
Everyday usage	15
CI/CD (GitHub Actions)	15
Troubleshooting quick hits	16
Appendix A • Version Pin Reference	17
Appendix B • The Generated Dockerfile	18
Appendix C • Sources & References	19

defoogi — created by Chris Osborn (@fozztexx) — is a single Docker container that bundles every compiler, assembler, library, and disk-image utility needed to build FujiNet firmware, libraries, and applications across every retro platform FujiNet supports — plus the modern embedded toolchains (PlatformIO / ESP32, Pico SDK) that the FujiNet hardware itself is built with.

You use it as a **command prefix**. Instead of installing a dozen cross-compilers on your machine, you run:

A terminal window with a dark background and light green text. The title bar shows three colored dots (red, yellow, green) and the text 'bash'. The terminal contains three lines of commands: '\$ defoogi make # run the project Makefile in-container', '\$ defoogi cc65 hello.c # or invoke one tool directly', and '\$ defoogi cmoc program.c'.

```
$ defoogi make # run the project Makefile in-container
$ defoogi cc65 hello.c # or invoke one tool directly
$ defoogi cmoc program.c
```

...and the build happens inside the container, but the artifacts land in your working directory **owned by you**, not by `root`.

The problem it solves

Building software for 8-bit and 16-bit machines means assembling a zoo of toolchains that were each written by a different person, in a different decade, with a different build system, and with wildly different packaging stories:

- ▶ Some are in `apt` / Homebrew (cc65, nasm, cpmtools, mtools).
- ▶ Some are Java jars that need a JDK (AppleCommander).
- ▶ Some only build from source under autotools and *won't compile natively on Windows* (CMOC needs Cygwin).
- ▶ Some are Free Pascal programs that need `fpc` just to bootstrap (MADS, Mad Pascal).
- ▶ Some are two-phase self-hosting builds that “take forever” — Open Watcom v2’s Dockerfile comment literally says so.
- ▶ Some need a specific pre-release commit with non-default build flags (z88dk, built `-z` for special MSX options).

Getting all of these onto one machine — at *mutually compatible, reproducible versions* — and then getting the *same* set onto a teammate’s macOS laptop, a Windows box, and a CI runner, is the actual problem. defoogi turns that problem into `docker pull`.

The headline feature: ownership preservation

FROM THE README

Unlike many Docker build environments, **defoogi preserves file ownership and permissions**, so your build artifacts stay usable on the host without extra `chown`.

This is not cosmetic. The usual Docker-build complaint is that everything the container writes to a bind-mounted directory comes out `root:root`, and you spend your life running `sudo chown -R`. defoogi solves this at the entrypoint (see Chapter 3).

2 · HOW DEFOOGI IS BUILT (ARCHITECTURE)

defoogi is *not* one monolithic Dockerfile. It is a set of independent build stages that are assembled at build time by the `Makefile`. Understanding this explains both its reproducibility and why each tool is cleanly separable.

The three core stages and the components

The `Dockerfiles/` directory holds three **core** stages and one stage per **component** tool:



```
head.docker ← CORE · defines the tooling base image
final.docker ← CORE · final image preamble (PlatformIO ...)
tail.docker ← CORE · installs all .debs, env, entrypoint
```

```
cc65 · cmoc · z88dk · open-watcom-v2 · mads · nasm
applecommander · atari-tools · dir2atr · cc1541
cpmtools · pico-sdk ← COMPONENTS, one tool each → a .deb
```

The `Makefile` classifies them programmatically:

```
CORE = head final tail
COMPONENTS = $(filter-out $(CORE),$(notdir $(DOCKERFILES:.docker=)))
```

The base image – `head.docker`

Everything starts from **Debian 13.0** (`ARG BASE=debian:13.0`) plus a minimal toolchain every component shares. Each component stage begins `FROM tooling AS <name>`, inheriting the same compiler base:

```
FROM ${BASE} AS tooling
RUN apt-get install -y --no-install-recommends \
    build-essential ca-certificates git
```

The clever part – build once, package as a `.deb`

Each component **builds its tool from source (or downloads it), installs it into a clean prefix, then wraps that prefix in a Debian package** inside the build stage. The pattern, from `cc65.docker`:

```
FROM tooling AS cc65
RUN git clone .../cc65.git && cd cc65 \
    && git checkout ${CC65_VERSION} \
    && PREFIX=/usr/local make \
    && PREFIX=/opt/cc65 make install
# ...then turn /opt/cc65 into /tmp/cc65.deb via dpkg-deb --build
```

So each stage's *output artifact* is a single file: `/tmp/<name>.deb`.

Assembly – the `Makefile` streams everything into one Dockerfile

This is the trick that ties it together. The `docker-build` target **streams** the core + component Dockerfiles into `docker build -f -` (read from stdin), injecting generated `COPY --from=` lines that pull each component's `.deb` out of its build stage:

```
docker-build:
    printf "%s\n" $(COMPONENTS) | \
    sed 's,.*,COPY --from=& /tmp/&.deb /tmp/packages/, ' | \
    cat head.docker <components> final.docker - tail.docker | \
    docker build -f - \
    $(shell sed 's/^\([^=]*\)=/--build-arg \1_VERSION=/' versions.env) ...
```

The result (written to `/tmp/defoogi.dockerfile`) is one big multi-stage Dockerfile where:

1. `head` builds `tooling`.
2. Every component stage builds its tool as a `.deb` — **these run in parallel** under BuildKit, each isolated.
3. `final` (a fresh `FROM tooling`) installs the no-compile-needed pieces: **PlatformIO** and `abimap` via `pipx`, plus `cmake` / `libmbedtls` / `libexpat`.
4. The generated `COPY --from=<component>` lines copy every built `.deb` into the final image.
5. `tail` installs them all at once (`apt-get install ./tmp/packages/*.deb`), adds runtime utilities, sets environment variables, creates the unprivileged `wario` user, and installs the entrypoint.

Reproducibility – `versions.env`

Every tool version is **pinned** in `versions.env`, and the `Makefile` turns each line into a `--build-arg <NAME>_VERSION=...`. The intent is stated in the `Makefile` itself:

WHY VERSIONS ARE PINNED

“...a stable, reproducible toolchain that can be matched to specific FujiNet firmware/software releases. In the future, older defoogi versions can still be rebuilt against the exact tool versions they were originally developed with.”

This is the single most important architectural property: **defoogi v1.4.6 is a fixed point** — a known-good constellation of 19 tool versions known to build FujiNet together. (See Appendix A.)

Multi-arch

`make multi-arch` `NAMESPACE=fuzztexx/` builds and pushes per-architecture tags (`-amd64`, `-arm64`); `make manifest` stitches them into one multi-arch manifest with `docker buildx imagetools create`. That is why the published image runs natively on both Intel and Apple-Silicon / ARM hosts.

Three scripts implement the “just prefix your command” experience: `start` (installed as `defoogi`), `cntnr-init` (the in-container entrypoint), and `defoogi-make` (a `make` convenience wrapper).

`start` → installed as `defoogi`

You copy `start` into your `PATH` renamed to `defoogi` (`make install` does `cp start /usr/local/bin/defoogi`). Because the script keys off its own name (`IMAGE=$(basename $0):latest`), the executable name *is* the image name.

When you run `defoogi make`, `start` issues roughly:

```
docker run --privileged -v /dev:/dev \
  --cap-add=SYS_ADMIN --cap-add SYS_PTRACE \
  -e DISPLAY -v ${HOME}/.Xauthority:/home/wario/.Xauthority --net=host \
  --rm -e HOSTDIR="${PWD}" -v "${PWD}":/workspace \
  fozztexx/defoogi:latest make
```

Notable bits:

- `-v "${PWD}":/workspace` — your current directory becomes `/workspace` inside.
- `-e HOSTDIR="${PWD}"` — tells the container the *real* host path, so it can recreate it (see below).
- X11 passthrough (`DISPLAY`, `.Xauthority`, `--net=host`) — so GUI tools (e.g. AppleCommander’s GUI) can display.
- `--privileged` + `SYS_ADMIN` — needed because the entrypoint performs a `mount --bind`.
- `--rm` for one-shot runs; `--daemon` keeps it running; `--shell` / `--super-shell` give an interactive shell as the workspace user / root.

`cntnr-init` — the ownership-preservation magic

This is the entrypoint, and where the “no root-owned artifacts” promise is kept. On startup it:

1. Reads the **owner uid:gid of** `/workspace` (your mounted directory).
2. If that owner is a normal user (`uid ≥ 1000`), it `usermod` / `groupmod` **s the in-container wario user to match your uid:gid** — so files `wario` writes are written as *you*.
3. **Bind-mounts** `/workspace` **back onto the original host path** (`HOSTDIR`) inside the container, then `cd` s there — so relative paths, `realpath`, and recorded absolute paths match the host. (This is why `--privileged` / `SYS_ADMIN` are required.)
4. Drops privileges and runs your command as `wario` via `sudo -u wario -E --preserve-env=PATH -H env "$@"`.

That uid/gid remap is the whole trick: artifacts come out owned by you, on Linux, with zero `chown`.

NOTE

On Docker Desktop for macOS / Windows the file-sharing layer already remaps ownership, so the benefit is most visible on native Linux Docker.

`defoogi-make` — directory-aware `make`

A thin wrapper so `make -C some/subdir` works through the container: it extracts the `-C` / `--directory` argument, mounts the *parent* directory, and re-invokes

`defoogi --directory <parent> make -C <target>` . Useful when your build references files one level up from the makefile.

Everything defoogi contains, grouped by job. Versions are the pinned values from `versions.env`.

Cross-compilers (high-level languages → retro CPUs)

Tool	Pin	CPU / target	What it is
cc65	6efe447	6502 / 65C02 / 65816	C compiler <i>plus</i> ca65 assembler, ld65 linker, ar65, da65. C64, Apple II, Atari 8-bit, NES, VIC-20, Oric... The backbone of FujiNet's 6502-family apps.
CMOC	0.1.97	6809 / 6309	C-like compiler for the Motorola 6809. Tandy CoCo, Dragon 32/64, OS-9. Requires lwtools.
lwtools	4.24	6809 / 6309	lwasm assembler + lwlink linker + lwar. CMOC emits assembly these turn into binaries.
z88dk	4c74585	Z80 / Z180 / 8080 / 8085	C compiler (zcc driving sccz80 and a bundled SDCC), assembler, linker, and a library covering 100+ Z80 machines: MSX, ZX Spectrum, CP/M, Amstrad, Coleco / ADAM. Built <code>./build.sh -z</code> for special MSX options.
Open Watcom v2	2025-08-02	8086 / 286 / 386 (x86)	Full C/C++ compiler + linker (wcc , wcl , wlink) for DOS, 16-bit Windows, OS/2, CP/M-86. The FujiNet MS-DOS / PC toolchain. Self-hosting two-phase build.
Mad Pascal	23e4c5f	6502 (Atari 8-bit)	Turbo-Pascal-compatible compiler, paired with MADS as its back-end. Ships FujiNet Pascal libs (fn_cookies , fn_tcp) — defoogi patches them at build time.

FujiNet-specific Z80 libraries layered into z88dk:

Library	Pin	Purpose
eoslib	70d476b	Coleco ADAM EOS C library — eos.lib + eos.h . (tschak909)
smartkeylib	1.1	ADAM SmartKeys function-key library. (tschak909)

Assemblers

Tool	Pin	CPU	Notes
ca65	in cc65	6502 family	cc65's macro assembler.
lwasmb	in lwtools	6809	lwtools' assembler.
MADS	2370bf0	6502 / Atari	Powerful Atari-centric 6502 macro assembler — the Atari-8-bit standard. Built from Pascal via <code>fpc</code> .
atasm	V1.30	6502 / Atari	MAC/65-compatible Atari assembler (built inside <code>dir2atr.docker</code>).
nasm	e9fac2f	x86	The Netwide Assembler — modern x86 / x86-64 assembly.

Disk-image & file tools – making the target media

The part most “just install a compiler” guides forget: a compiled binary is useless until it is inside a disk image the target machine (or FujiNet) can mount. defoogi bundles a tool for **every** disk format FujiNet platforms use.

Tool	Pin	Format	Platform
atari-tools (<code>atr</code>)	835d5a6	<code>.ATR</code> read/write/extract	Atari 8-bit
dir2atr (AtariSIO)	bbccb15	build <code>.ATR</code> from a directory	Atari 8-bit
cc1541	4.2	<code>.D64</code> (1541 floppy image)	Commodore 64 / VIC-20
AppleCommander (<code>ac</code> , <code>acx</code>)	12.0	<code>.dsk</code> <code>.do</code> <code>.po</code> <code>.2mg</code> , ShrinkIt	Apple II
cpmtools	2.23	CP/M filesystems	CP/M + Coleco ADAM
mttools	apt	FAT12/16 floppy & disk images	MS-DOS / PC
decb (Toolshed)	v2_4_2	Disk Extended Color BASIC <code>.dsk</code>	Tandy CoCo

ADAM DETAIL WORTH KNOWING

`cpmtools.docker` writes a custom `/usr/local/share/diskdefs` defining `colemco-adam` (5.25", 40 tracks) and `colemco-adam-3.5` (3.5", 160 tracks) geometries — so you can build ADAM media that stock `cpmtools` doesn't know about out of the box.

Embedded / FujiNet-hardware toolchains

Tool	Pin	Target	Role
PlatformIO	latest	ESP32 (ESP-IDF / Arduino)	The FujiNet firmware build system itself — how the device firmware is compiled and flashed.
abimap	latest	—	ABI / symbol-version map helper used in some build flows.
Pico SDK	2.2.0	RP2040 (Raspberry Pi Pico)	SDK at <code>/usr/local/share/pico-sdk</code> (<code>PICO_SDK_PATH</code> preset) for RP2040-based FujiNet peripherals / variants.

Tool	Pin	Target	Role
picotool	w/ pico-sdk	RP2040	Inspect / flash RP2040 binaries.

Base system, libraries & build dependencies (the invisible glue)

These come from `head`, `final`, `tail`, and from being pulled in as dependencies. They're "free" inside defoogi but each is something you'd otherwise have to provide yourself:

- ▶ **Debian 13.0** userland + **build-essential** (gcc/g++/make), **git**, **ca-certificates**, **cmake**.
- ▶ **Free Pascal Compiler (fpc)** — required to build MADS / Mad Pascal.
- ▶ **default-jdk (Java)** — pulled in as an AppleCommander dependency; required to run `ac` / `acx`.
- ▶ **SDCC** — built *as part of* z88dk (`BUILD_SDCC=1`).
- ▶ A pile of **Perl modules** (Capture::Tiny, Clone, Path::Tiny, YAML, Modern::Perl...) plus `bison`, `flex`, `ragel`, `re2c`, `m4`, `ccache`, `texinfo` — all required just to build z88dk.
- ▶ Runtime utilities in `tail`: `mtools`, `curl`, `wget`, `file`, `jq`, `xxd`, `zip` / `unzip`, `less`, `bsdmainutils`, `libz-dev`, `sudo`.
- ▶ Preset environment: `WATCOM=/opt/watcom`, `PATH+=$WATCOM/bin`, `PLATFORMIO_CORE_DIR=/workspace/.platformio`, `PICO_SDK_PATH=/usr/local/share/pico-sdk`.

Declared-but-not-built (housekeeping note)

NOTE

Two entries are referenced but **not currently produced** by any Dockerfile in this revision. `versions.env` pins `VASM=2_0c`, but there is no `vasm.docker`; and `tail.docker` exports `VBCC=/opt/vbcc` (adding `$VBCC/bin` to `PATH`) with no `vbcc.docker` to build it and no `VBCC` pin. These look like scaffolding for a planned **vasm / vbcc** (Amiga / 68k-style) toolchain. They're harmless today but worth flagging so you don't go hunting for `vasmm68k`.

What if you *didn't* use defoogi? Here is what it actually takes to stand up each tool on **Linux**, **Windows**, and **macOS**. The pattern that emerges: a few tools are well-packaged everywhere, but the majority are source-only, several **do not build natively on Windows** (Cygwin / MSYS required), and **none** of the platform combinations gets you a pinned, mutually-compatible set without manual version juggling.

Legend: OK packaged / one command SRC build from source / caveats NO no native path (needs Cygwin / MSYS / VM)

cc65 (6502)

OS	Method
Linux OK	<code>apt install cc65</code> (Debian / Ubuntu); openSUSE Build Service RPM/DEB.
macOS OK	<code>brew install cc65</code> .
Windows OK	Official <code>.exe</code> snapshot installer (sets env vars) or unzip the binary snapshot.

cc65 is the easy one. Caveat: distro packages can lag the upstream `git HEAD` FujiNet relies on, so you may still `git clone && make` to match `versions.env`'s `6efe447`.

CMOC + lwtools (6809)

OS	Method
Linux SRC	Build both from source. lwtools: <code>tarball</code> , <code>make && make install</code> . CMOC: <code>./configure && make && make install</code> (needs <code>flex / yacc</code>). <code>lwasm / lwlink</code> must be on <code>PATH</code> .
macOS SRC	Same source build — CMOC supports Darwin, but no Homebrew formula.
Windows NO	Per the author, CMOC cannot be compiled as a native Windows app — you must use Cygwin (lwtools likewise). A third-party WinCMOC bundle exists but lags far behind and isn't the pinned <code>0.1.97</code> .

defoogi additionally builds `decb` from Toolshed here (CoCo `.dsk` handling) — another from-source step you'd have to replicate.

z88dk + SDCC (Z80)

OS	Method
Linux SRC	Nightly source tarball or <code>git clone --recursive + ./build.sh</code> ; a snap exists but won't be the pinned commit or carry the <code>-z MSX</code> flags.
macOS SRC	Nightly <code>z88dk-osx-latest.zip</code> exists, but to match the pinned pre-release commit you build from source.
Windows NO	Nightly <code>z88dk-win32-latest.zip</code> exists, but building the classic libs from source requires MSYS or Cygwin .

Then you'd *still* clone & build **eoslib** and **smartkeyslib** into z88dk's lib tree — exactly what `z88dk.docker` does — and satisfy its long Perl-module + bison/flex/ragel/re2c build wall.

Open Watcom v2 (x86 / DOS)

OS	Method
Linux OK	Download the portable snapshot zip (<code>ow_portable_v2_stable.zip</code>); set <code>WATCOM / PATH / INCLUDE</code> . Or build from source (two-phase, slow).
macOS SRC	Portable snapshot / build from source; set env vars manually.
Windows OK	Official installer from the snapshot builds.

Workable, but you wire up three environment variables yourself — defoogi presets them. The source build “takes forever” (the Dockerfile’s own words).

MADS + Mad Pascal (Atari 6502)

OS	Method
Linux SRC	Install <code>fpc</code> , then <code>fpc -Mdelphi mads.pas</code> and <code>fpc src/mp.pas</code> from the Mad-Assembler / Mad-Pascal repos.
macOS SRC	Same — install Free Pascal, build from source.
Windows OK	Prebuilt Windows binaries published by the author (tebe6502); otherwise install FPC and build.

You’d also replicate defoogi’s library staging (copying `base/lib/blibs/dlibs` , upper-case symlinks) and the FujiNet `fn_cookies` / `fn_tcp` source patch.

Atari disk tooling, atari-tools, cc1541

Tool	OS	Method
atasm / dir2atr	Lin/mac SRC	atasm: <code>clone & make .</code> dir2atr: <code>clone AtariSIO (make tools && make tools-install)</code> ; needs <code>libncurses-dev</code> . AtariSIO is Linux-centric.
atasm / dir2atr	Win SRC	Hias publishes Windows AtariSIO builds; atasm under MSYS/MinGW.
atari-tools (atr)	all SRC	<code>git clone jhallen/atari-tools && make .</code> No package.
cc1541	all SRC	Clone from Bitbucket, <code>make && make install</code> . Community packages exist but versions vary.

AppleCommander (Apple II disks)

OS	Method
All three SRC	Download cross-platform jars from GitHub releases — but you must have Java 11+ installed, and you’ll want to write <code>ac / acx</code> shell-script wrappers (<code>java -jar ...</code>) yourself. defoogi does both, and pulls <code>default-jdk</code> automatically.

cpmtools / mtools / nasm

Tool	OS	Method
cpmtools/mtools	Linux OK	<code>apt install cpmtools mtools .</code>

Tool	OS	Method
cpmtools/mttools	macOS OK	<code>brew install cpmtools mttools</code> .
cpmtools/mttools	Windows SRC	Cygwin build (cpmtools); mttools via Cygwin/MSYS.
nasm	all OK	<code>apt install nasm</code> · <code>brew install nasm</code> · official Windows installer.

Even here, you'd add defoogi's custom Coleco ADAM `diskdefs` by hand to make ADAM media.

PlatformIO + Pico SDK (embedded)

Component	Method
PlatformIO OK	<code>pipx install platformio</code> — cross-platform, genuinely easy everywhere.
Pico SDK SRC	Clone <code>pico-sdk</code> + submodules, set <code>PICO_SDK_PATH</code> , install the <code>gcc-arm-none-eabi</code> cross toolchain + <code>newlib</code> ; build picotool from source (needs <code>libusb</code> , <code>cmake</code> , <code>pkg-config</code>).

The DIY pain summary

Tool	Linux	Windows	macOS	Packaged?
cc65	OK	OK	OK	<code>apt</code> · <code>brew</code> · <code>exe</code>
nasm	OK	OK	OK	<code>apt</code> · <code>brew</code> · <code>exe</code>
cpmtools / mttools	OK	SRC	OK	<code>apt</code> · <code>brew</code>
Open Watcom v2	OK	OK	SRC	snapshot / installer
AppleCommander	SRC	SRC	SRC	jar (needs JDK 11)
PlatformIO	OK	OK	OK	<code>pipx</code>
z88dk (+eoslib/keys)	SRC	NO	SRC	nightly / snap (unpinned)
CMOC + lwtools	SRC	NO	SRC	source only
MADS / Mad Pascal	SRC	OK	SRC	source (Win binaries)
atasm / dir2atr / atr	SRC	SRC	SRC	source only
cc1541	SRC	SRC	SRC	source (mostly)
Pico SDK + picotool	SRC	SRC	SRC	source + ARM toolchain

Tally: of 13 tool groups, only **three** (cc65, nasm, PlatformIO) are genuinely one-command everywhere. **Two won't build natively on Windows at all.** And **none** of this gives you version pins matched to a FujiNet release.

The matrix above shows the *breadth* problem. Here is why the container wins on the dimensions that matter for FujiNet development:

1. **One install, not thirteen.** `docker pull fozztexx/defoogi` (or one local build) replaces a multi-day setup spanning apt, Homebrew, pipx, Java, Free Pascal, Cygwin / MSYS, and a half-dozen `git clone && make` builds — *per developer, per machine*.
2. **Reproducibility pinned to FujiNet releases.** `versions.env` freezes 19 components at exact commits/tags. A given defoogi tag is a *known-good constellation*. DIY installs drift to slightly different upstream versions and “works on my machine” bugs follow. defoogi can be **rebuilt from an old tag** to reproduce a historical build exactly.
3. **True cross-platform parity.** The container is byte-for-byte identical on Linux, Windows (Docker Desktop / WSL2), and macOS (incl. Apple Silicon via the multi-arch manifest). The DIY path differs on every OS — and on Windows several tools require Cygwin / MSYS that subtly change behavior.
4. **The disk-image tools are included and pre-configured.** Compilers are only half the job; you need `atr`, `dir2atr`, `cc1541`, `ac/acx`, `cpmtools` (with **ADAM diskdefs**), `mttools`, and `decb` to produce mountable media. defoogi ships all of them, wired up — including config a fresh install wouldn’t have.
5. **All the build glue is solved.** Java for AppleCommander, FPC for MADS, SDCC inside z88dk, the Perl-module wall z88dk needs, the FujiNet `fn_cookies` / `fn_tcp` patches, the eoslib / smartkeyslib staging, `WATCOM` / `PICO_SDK_PATH` env vars — all pre-resolved.
6. **Ownership preservation.** Artifacts come out owned by you (uid/gid remap + bind-mount) — no `sudo chown` ritual, no root-owned junk in your repo.
7. **Local and CI are the same environment.** The exact image used on your laptop runs in GitHub Actions, enabling the **edit → push → CI build → mount over HTTP via FujiNet → boot** workflow. DIY means maintaining two parallel setups and keeping them in sync.
8. **Firmware and apps in one place.** It is the only environment covering both the **device firmware** (PlatformIO / ESP32, Pico SDK) and **every client-platform app toolchain** — so firmware devs and app devs share one tool.

THE HONEST COUNTERPOINT

defoogi requires Docker; the first build / pull is large; `--privileged` is needed for the bind-mount trick; and GUI / serial-flashing workflows rely on the X11 + `/dev` passthrough the `start` script sets up. For the FujiNet use case, those costs are small next to maintaining a dozen hand-built cross toolchains across three operating systems.

Install

```
# 1. Get Docker (the repo even ships get-docker.py for Debian-family Linux)
./get-docker.py          # adds Docker's apt repo, installs docker-ce,
                          # and adds you to the docker group

# 2. Put the launcher on your PATH, named `defoogi`
sudo make install        # cp start /usr/local/bin/defoogi
# or: cp start ~/bin/defoogi && chmod +x ~/bin/defoogi

# 3. first run pulls fozztexx/defoogi:latest automatically
```

To **build the image yourself** (reproducible from the pins):

```
make          # assembles + builds defoogi:1.4.6 and defoogi:latest
make rebuild  # --no-cache --pull, full clean build
```

Everyday usage

```
cd ~/my-fujinet-app

defoogi make          # run the project Makefile in-container
defoogi cc65 hello.c  # 6502
defoogi cmoc program.c # 6809
defoogi zcc +cpm program.c # z88dk for CP/M
defoogi wcl -bcl=dos prog.c # Open Watcom for DOS

# Disk images:
defoogi atr disk.atr format ; defoogi atr disk.atr write prog.xex
defoogi cc1541 -f prog -w prog.prg disk.d64
defoogi ac -as disk.dsk PROG < prog.bin

# Interactive shells:
defoogi --shell          # shell as `wario` (you), in your workspace
defoogi --super-shell    # root shell (for poking at the image)
```

CI/CD (GitHub Actions)

Use the same image as a container step so CI builds bit-identically to local, then publish the artifact as a downloadable asset that FujiNet can mount over HTTP. The promised loop: **edit → push → CI builds with defoogi → boot retro machine → mount the build directly via FujiNet → run**. No floppies, no SD-card sneakernet.

Troubleshooting quick hits

Symptom	Cause / fix
No <code>/workspace</code> directory found	You ran the raw container without the <code>start / defoogi</code> wrapper. Use <code>defoogi ...</code> (it sets <code>-v \$PWD:/workspace</code> and <code>HOSTDIR</code>).
Artifacts owned by a system user	<code>/workspace</code> is owned by a uid < 1000, so <code>cntnr-init</code> deliberately skips the remap and warns. Run from a normally-owned directory.
GUI tool won't display	Ensure <code>\$DISPLAY</code> is set and <code>~/.Xauthority</code> exists; start forwards both with <code>--net=host</code> .
<code>vasm</code> / <code>vbcc</code> not found	Expected — declared in <code>versions.env</code> / <code>tail</code> but not built in this revision (\$4).

APPENDIX A · VERSION PIN REFERENCE

Every pin in `versions.env`, with what it controls. defoogi tag at time of writing: **1.4.6**.

Key	Value	Controls	Built by
AC	12.0	AppleCommander jars	applecommander
ATARISIO	bbccb15	dir2atr (AtariSIO tools)	dir2atr
ATARIT00LS	835d5a6	atr (atari-tools)	atari-tools
ATASM	V1.30	atasm assembler	dir2atr
CC1541	4.2	cc1541 (D64)	cc1541
CC65	6efe447	cc65 suite	cc65
CMOC	0.1.97	CMOC 6809 compiler	cmoc
CPMT00LS	2.23	cpmtools (+ ADAM diskdefs)	cpmtools
EOSLIB	70d476b	ADAM EOS lib (in z88dk)	z88dk
LWT00LS	4.24	lwasm / lwlink (for CMOC)	cmoc
MADSASM	2370bf0	Mad-Assembler	mads
MADSPAS	23e4c5f	Mad-Pascal	mads
OW2	2025-08-02-Build	Open Watcom v2	open-watcom-v2
PICOSDK	2.2.0	Pico SDK + picotool	pico-sdk
SMARTKEYS	1.1	ADAM SmartKeys lib (in z88dk)	z88dk
TOOLSHED	v2_4_2	decb (CoCo .dsk)	cmoc
VASM	2_0c	<i>declared, not built</i>	—
Z88DK	4c74585	z88dk + SDCC	z88dk
NASM	e9fac2f	nasm	nasm

Not in `versions.env` (installed at latest): **PlatformIO**, **abimap**, **mttools**, **default-jdk (Java)**, **fpc**, **SDCC** (built within z88dk), **cmake**.

APPENDIX B • THE GENERATED DOCKERFILE

What the `Makefile` feeds to `docker build` (written to `/tmp/defoogi.dockerfile`), conceptually:

```
[ head.docker ]           # FROM debian:13.0 AS tooling
[ cc65.docker ]           # FROM tooling AS cc65   → /tmp/cc65.deb
[ cmoc.docker ]           # FROM tooling AS cmoc   → /tmp/cmoc.deb
[ z88dk.docker ]          # ...one stage per component, in parallel
[ ... every other component ... ]
[ final.docker ]          # FROM tooling (final image; PlatformIO)
COPY --from=cc65 /tmp/cc65.deb /tmp/packages/      # ← generated by sed
COPY --from=cmoc /tmp/cmoc.deb /tmp/packages/
COPY --from=...           # one COPY per component
[ tail.docker ]           # apt-get install /tmp/packages/*.deb;
                          # runtime tools; env; create `wario`;
                          # ENTRYPOINT cntnr-init
```

Build-args are injected from `versions.env` (`AC=12.0` → `--build-arg AC_VERSION=12.0`), plus `MAINTAINER` and `WSUSER`.

defoogi itself

- ▶ defoogi repository — github.com/tschak909/defoogi (analyzed locally at `~/Workspace/defoogi`, tag 1.4.6)
- ▶ Docker Hub — hub.docker.com/repository/docker/fozztexx/defoogi
- ▶ FujiNet — fujinet.online

Toolchains

- ▶ cc65 — cc65.github.io · Homebrew formula · NESdev “Installing CC65”
- ▶ CMOC — perso.b2b2c.ca/~sarrazip/dev/cmoc.html (+ manual) · lwtools-on-Cygwin (Sub-Etha Software)
- ▶ lwtools — lwtools.ca
- ▶ z88dk — z88dk.org · installation wiki · nightly.z88dk.org
- ▶ Open Watcom v2 — openwatcom.org · github.com/open-watcom/open-watcom-v2 (build/server/install.txt)
- ▶ MADS / Mad-Pascal — github.com/tebe6502/Mad-Assembler · .../Mad-Pascal
- ▶ atasm — github.com/CycoPH/atasm
- ▶ AtariSIO (dir2atr) — github.com/Hiassoft/AtariSIO
- ▶ atari-tools — github.com/jhallen/atari-tools
- ▶ cc1541 — bitbucket.org/ptv_claus/cc1541
- ▶ AppleCommander — applecommander.github.io · releases · install (Java 11)
- ▶ cpmtools — moria.de/~michael/cpmtools · Homebrew
- ▶ mtools — gnu.org/software/mtools
- ▶ nasm — nasm.us · Homebrew
- ▶ Toolshed (decb) — github.com/nitros9project/toolshed
- ▶ eoslib / smartkeyslib — github.com/tschak909/eoslib · .../smartkeyslib
- ▶ PlatformIO — platformio.org
- ▶ Pico SDK / picotool — github.com/raspberrypi/pico-sdk · .../picotool

Generated for the FujiNet Manuals Project. Verified against defoogi v1.4.6 source. Upstream packaging behavior reflects the state at June 2026 and may change.